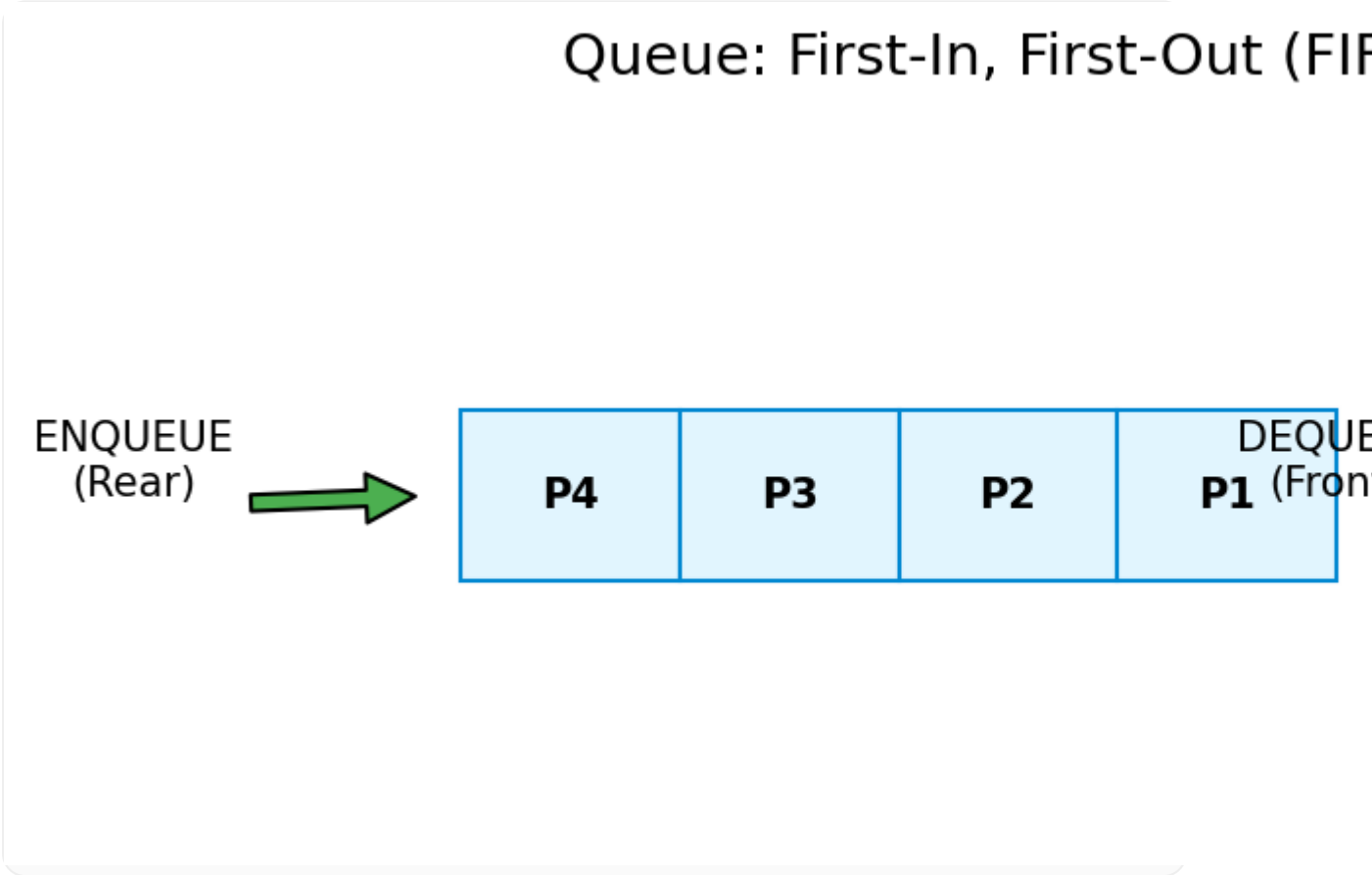
Queue & Deque in Python

FIFO Principles and Double-Ended Operations

1. The FIFO Concept

A **Queue** is a linear structure following **First-In, First-Out**. The first element added is the first one removed. Think of a line at a ticket counter.



- **Enqueue:** Add to Rear (using append) O(1)
- **Dequeue:** Remove from Front (using `popleft`) O(1)

2. Double-Ended Queue (Deque)

A **Deque** allows insertion and deletion from both ends. Python's `collections.deque` is the standard implementation.

3. Detailed Syntax: `collections.deque`

```
from collections import deque

q = deque()

# Standard Queue (FIFO)
q.append("A")      # Right end
q.append("B")
q.popleft()           # Returns "A" (FIFO)

# Double-Ended Usage
q.appendleft("Z") # Insert at front
q.pop()               # Remove from back
```

> **Tech Tip:** Python's `list.pop(0)` is **O(n)** because it re-indexes every element. Always use `deque.popleft()` for **O(1)** performance.

4. Problem 1: Implement Stack using Queues

To implement LIFO (Stack) using FIFO (Queue), we rotate the queue on every push so the newest element is always at the "head" (front) of the queue.

```
class MyStack:
    def __init__(self):
        self.q = deque()

    def push(self, x: int):
        self.q.append(x)
        # Re-order the entire queue
        for _ in range(len(self.q) - 1):
            self.q.append(self.q.popleft())

    def pop(self) -> int:
        return self.q.popleft()
```

5. Problem 2: Number of Recent Calls

A "sliding window" problem. We only keep timestamps within `[t-3000, t]`.

```python
class RecentCounter:
    def __init__(self):
        self.records = deque()

    def ping(self, t: int) -> int:
        self.records.append(t)

        # Remove anyone outside the 3000ms window
        while self.records[0] < t - 3000:
            self.records.popleft()

        return len(self.records)
```